



Kimi K3, The Manos, The Mythos, The Legendos

Архитектура Kimi K3: сжатая память, внимание к глубине, скрытая экспертная маршрутизация и высокая производительность

КИМБО ЧЕН, ШУБХАМ ЧОУДХАРИ, БРАЙАН ШАН, И ДИЛАН ПАТЕЛЬ

3 АВГУСТА 2026 ГОДА ОПЛАЧЕННЫЙ

Модель Kimi K3 произвела фурор сразу после анонса, возглавив списки лидеров, зарекомендовав себя как модель с открытым фронтиром. Сообщество стремится понять, как работает Kimi K3, но многих удивляют нетрадиционные методы, обеспечивающие ее эффективность. Эта статья поможет разобраться в основных принципах архитектуры модели Kimi K3.

Кими Дельта , Внимание

Kimi Delta Attention (KDA) — это слой линейного внимания в гибридном механизме внимания Kimi K3. Мы проследили происхождение KDA, начиная с линейного внимания, DeltaNet, Gated DeltaNet (GDN) и заканчивая KDA.

Линейное Внимание

Линейное внимание появилось в результате **исключения операции softmax** стандартного алгоритма внимания **softmax**. Ниже мы сравниваем формы итеративного вывода, которые показывают вычисление выходного вектора позиции токена t :

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

[Управление](#) [Отклонить](#) [Принять](#)

$$\mathbf{o}_t^{\text{softmax}} = \sum_{i=1}^t \frac{\exp(\mathbf{q}_t^\top \mathbf{k}_i)}{\sum_{j=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_j)} \mathbf{v}_i$$

$$\mathbf{o}_t^{\text{linear}} = \sum_{i=1}^t (\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{v}_i$$

Источник: [DeltaNet Explained \(Часть I\)](#)

Убрав операцию softmax, мы можем изменить порядок операций и снизить вычислительную сложность механизма внимания с квадратичной до линейно

$$\begin{aligned} \mathbf{o}_t &= \sum_{i=1}^t (\mathbf{q}_t^\top \mathbf{k}_i) \mathbf{v}_i \\ &= \sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top \mathbf{q}_t \\ &= \underbrace{\left(\sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top \right)}_{\mathbf{S}_t \in \mathbb{R}^{d \times d}} \mathbf{q}_t \\ &= \left(\underbrace{\sum_{i=1}^{t-1} \mathbf{v}_i \mathbf{k}_i^\top}_{\mathbf{S}_{t-1} \in \mathbb{R}^{d \times d}} + \mathbf{v}_t \mathbf{k}_t^\top \right) \mathbf{q}_t \end{aligned}$$

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d \times d}$$

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^d$$

Источник: [Линейное внимание и не только \(интерактивное руководство с Сонлином Янгом\)](#)

Векторы $\mathbf{q}$, $\mathbf{k}$, $\mathbf{v}$ имеют размерность L на d . Вычислительная сложность о уравнений равна $O(Ld^2)$, что делает вычисления линейными. Сравнивая н уравнения с уравнением softmax-внимания, мы видим, что **softmax-вним** требует доступа ко всем предыдущим векторам ключей и значений, в то в как **линейное внимание сжимает все предыдущие векторы ключей и значен** одно скрытое состояние $\mathbf{S}$.

Мы переосмысливаем новые уравнения как **цель онлайн-обучения**. рассматриваем матрицу $\mathbf{S}$ как ассоциативную память, в которой хранятся с между ключевым вектором $\mathbf{k}$ и вектором значений $\mathbf{v}$, и извлекаем $\mathbf{v}$, умножая $\mathbf{k}$. Тогда первое уравнение можно интерпретировать как **непрерывное обновл матрицы $\mathbf{S}$ в каждой позиции для улучшения извлечения**. Наконец, мы мс интерпретировать член $\mathbf{v}_t @ \mathbf{k}_t^\top$ как градиент функции потерь $-(\mathbf{S} @ \mathbf{k}_t^\top) @ \mathbf{v}_t$ отношению к $\mathbf{S}$.

$$\text{Objective: } \mathcal{L}_t(\mathbf{S}) = -\langle \mathbf{S} \mathbf{k}_t, \mathbf{v}_t \rangle$$

$$\begin{aligned} \text{SGD update: } \mathbf{S}_t &= \mathbf{S}_{t-1} - \beta_t \nabla \mathcal{L}_t(\mathbf{S}_{t-1}) \\ &= \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \end{aligned}$$

Дельтанет

С точки зрения онлайн-обучения, мы видим, что значения матрицы $\mathbf{S}$ неограниченно расти: по мере увеличения последовательности старая и н

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

DeltaNet улучшает линейное внимание за счет **изменения функции потерь минимизацию L2-нормы при извлечении значений**. В отличие от функции потерь линейного внимания, функция потерь DeltaNet регулирует рост S . Это приводит к появлению нового правила обновления матрицы S — правила дельта, показано ниже:

$$\text{Objective: } \mathcal{L}_t(S) = \frac{1}{2} \|S\mathbf{k}_t - \mathbf{v}_t\|^2$$

$$\begin{aligned} \text{SGD update: } S_t &= S_{t-1} - \beta_t \nabla \mathcal{L}_t(S_{t-1}) \\ &= S_{t-1} - \beta_t (S_{t-1}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top \end{aligned}$$

Источник: [«Линейное внимание и не только» \(интерактивное руководство Сонлином Янгом\)](#)

Правило дельта-распределения становится основой уравнения внимания DeltaNet:

$$S_t = S_{t-1} - \beta_t (S_{t-1}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

По сути, $S_{t-1} @ \mathbf{k}_t - \mathbf{v}_t$ представляет собой ассоциации, не имеющие отношения к текущему ключу и значению, и DeltaNet выполняет целенаправленное удаление таких ассоциаций.

Gated DeltaNet

GDN and KDA are adaptations of DeltaNet. Gated DeltaNet applies the LSTM forget gate *alpha* on the matrix S , allowing the model to control memory lifespan with weight decay. KDA further expands *alpha* into a diagonal matrix that enables fine-grained per-channel memory decay and positional awareness.

$$S_t^{\text{GDN}} = \alpha_t S_{t-1} - \beta_t (\alpha_t S_{t-1}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Thanks for reading SemiAnalysis! This post is public so feel free to share it.

FlashKDA Algorithm

Moonshot developed FlashKDA, their custom kernels for KDA, and [open-source](#). Here we explain the algorithm and derive the arithmetic intensity.

Algorithm

First, let's start from an alternative formulation of the recurrence formula:

```
u_t = beta_t * (v_t - (D_t @ S_{t-1}).T @ k_t)
S_t = D_t @ S_{t-1} + k_t @ u_t.T
o_t.T = q_t.T @ S_t
```

Here, D_t is the diagonal matrix of the alpha forget gate, and u_t is the delta in the delta rule. For decode, the kernel roughly follows the formula. For prefill, we parallelize the operation by unrolling the recurrence formula in chunks of tokens, order to efficiently execute the operations on GPUs. Assume we unroll token i to j , the starting state is S_{i-1} , we get:

```
S_j = D_{j:i} @ S_{i-1} + sum(D_{j:t+1} @ k_t @ u_t.T, t=i:j)
o_j.T = q_j.T @ S_j
      = q_j.T @ D_{j:i} @ S_{i-1} + sum(q_j.T @ D_{j:t+1} @ k_t @ u_t.T, t=i:j)
```

$D_{j:i}$ refers to the cumulative decay from token i to j : $D_j @ D_{j-1} @ D_{j-2} @ \dots @ D_i$. In FlashKDA's matrix form, the formula becomes:

```
S_out = D_{j:i} @ S_in + K_restore.T @ U
M_qk = tril(Q_decay @ K_inv.T)
```

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

- S_{out} refers to the state at the end position of a chunk
- $K_{restore}$ is the matrix form of $D_{j:t+1} @ k_t$
- Q_{decay} is the matrix form of $q_{j:T} @ D_{j:i}$
- $Q_{decay} @ K_{inv.T}$ the matrix form of $q_{j:T} @ D_{j:t+1} @ k_t$, derived from ($@ D_{j:i} @ (D_{t:i}^{-1} @ k_t)$)
- M_{qk} is the causal mask, so it's a lower triangular matrix

U is the matrix form of unrolled u_t . To compute this, we apply UT transform compute the following:

```
B = Diag(beta) @ (V - K_decay @ S_in)
L = StrictTril(Diag(beta) @ K_decay @ K_inv.T)
U = (I + L)^-1 @ B
```

Please consult [Songlin Yang's blog post](#) and [Kimi Linear paper section 3.1](#) for the derivation. Note that here U corresponds to the pseudo-value term in the Kimi L paper.

Implementation-wise, FlashKDA launches 2 kernels: K1 and K2. K1 prepares ch level tensors in parallel, including:

```
a = exp2(cumsum(g))
K_decay = Diag(a) @ K
Q_decay = Diag(a) @ Q
K_inv = Diag(a)^-1 @ K
K_restore = a[-1] * K_inv
L = StrictTril(Diag(beta) @ K_decay @ K_inv.T); INV = (I + L)^-1
M_qk = tril(Q_decay @ K_inv.T)
```

Here a is the cumulative decay, where each element is the cumulative decay at a t position.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

$$S = \text{Diag}(a[-1]) @ S + K_{\text{restore}}.T @ U$$

Complexity Analysis

Here we analyze the complexity of an attention head. For decode, the critical path computations are:

- $D_t @ S_{t-1}$: Element-wise multiplication, $D \times D$
- $S_{t-1}.T @ k_t$: $D \times D \times 1$
- $k_t @ u_t.T$: $D \times 1 \times D$
- $q_t.T @ S_t$: $1 D \times D$

The decode kernel roughly performs $7 \times D^2$ FLOPs.

Reading and writing the FP32 recurrent state dominates the memory traffic, so the memory traffic is roughly $8 \times D^2$ bytes.

For prefill, the critical path of K1 is at computing L, INV, and M_qk.

- L: $C \times D \times C$
- INV: [Neumann factorization](#), performs 6 $C \times C \times C$ matrix multiplications
- M_qk: $C \times D \times C$

For K2,

- K_decay @ S: $C \times D \times D$
- Q_decay @ S: $C \times D \times D$
- M_qk @ U: $C \times C \times D$
- INV @ B: $C \times C \times D$

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

For memory traffic:

- K1 read Q, K, g: $C \times D$
- K1 write and K2 read Q_decay, K_decay, K_restore: $C \times D$
- K1 write and K2 read INV, M_qk: $C \times C$
- K2 read V and write O: $C \times D$
- K2 read and write S once per kernel: $D \times D$

In total, FlashKDA accesses $3 * 2 * C * D + 2 * (3 * 2 * C * D + 2 * 2 * C * C) + 2 * 2 * C * D = 8 * 2 * C * D$ bytes. At the kernel level, it accesses $T/C * (8 * C^2 + 22 * C * D) + 8 * D^2 \sim O(TC + D^2)$.

This concretely shows that the computational complexity of KDA:

- Prefill: Linear to sequence length for both computation and memory
- Decode: Constant to sequence length for both computation and memory

Kimi Linear

Moonshot trained Kimi Linear models as proof of concept for their KDA design, so we can infer Kimi K3's architecture design from Kimi Linear. Comparing the K3 re tech blog with Kimi Linear, we see Kimi K3 shares the shared expert count, the h linear attention ratio, and the general attention module design.

Политика использования cookie-файлов

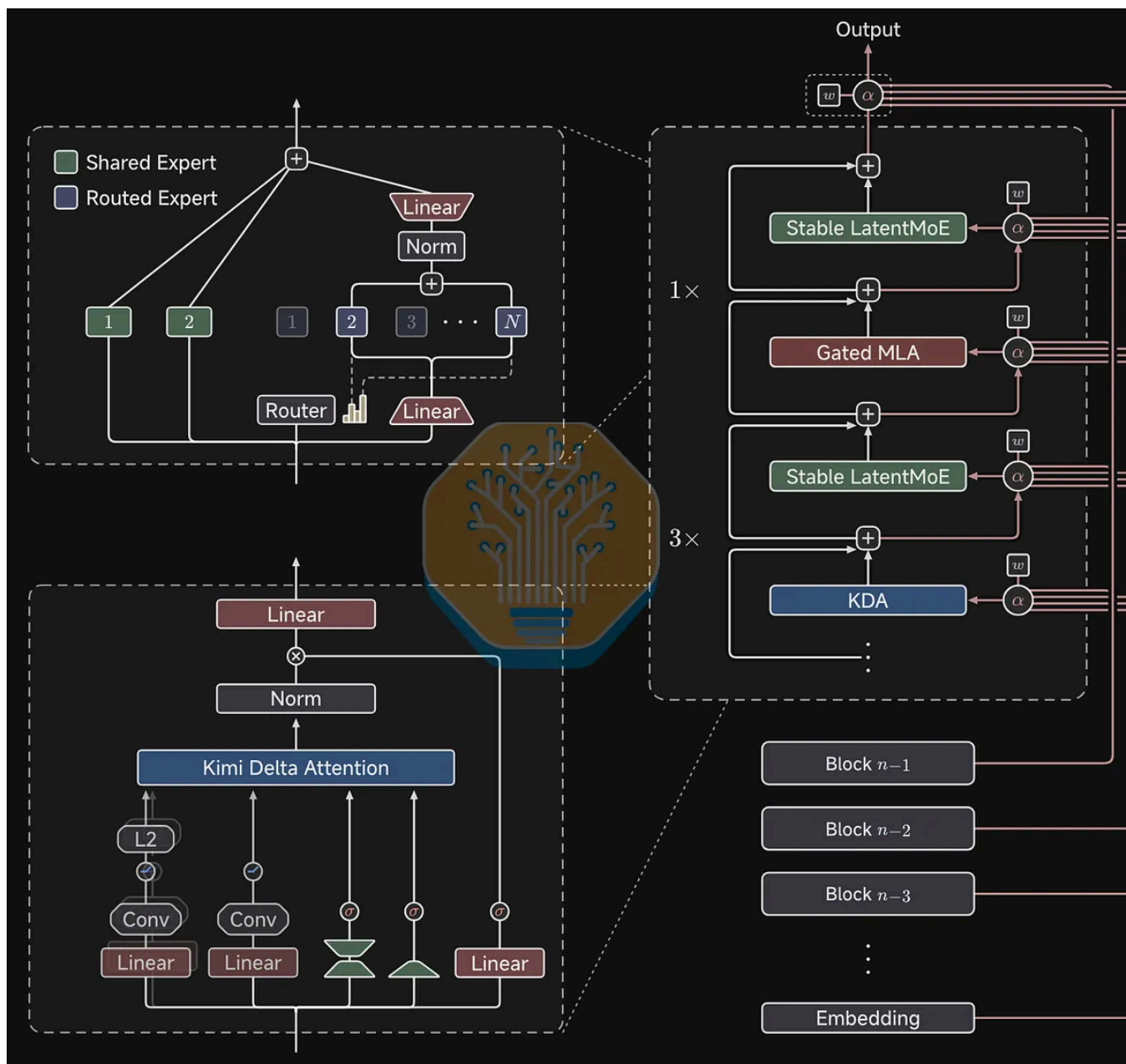
Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Config	Kimi Linear	Kimi K3
Num Activated Experts	9 (1 shared 8 routed)	18 (2 shared 16 routed)
Num Total Routed Experts	256 (sparsity 32)	896 (sparsity 56)
KDA to MLA Ratio	3:1	3:1
Activation Function	Swish	Sigmoid Tanh Unit (SiTU)
KDA Output Gate Type	Low rank projection	Linear transformation with k
Full Attention Type	MLA	Gated MLA
MoE Type	MoE	Stable LatentMoE
Token Routing	Sigmoid + bias	Quantile balancing
Optimizer	MuonClip	Per-head Muon

Source: SemiAnalysis

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).



Source: [Kimi K3 Tech Blog](#)

The diagram above shows the operations performed on the inputs of KDA. For query, key, and value, we apply linear transformation and short convolution. Applying short convolution effectively captures local token dependencies, and doing a padding convolution avoids breaking causality. We additionally apply L2 norm to query and key to stabilize the eigenvector of the transition and the output matrix. For the decay memory gates, α is a low rank projection, and β is a

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Kimi Linear interleaves KDA with full attention Multi-head Latent Attention (MLA). Kimi Linear showed that 3:1 is the ideal KDA to MLA ratio that balances performance and efficiency. KDA also serves as a strong position-aware operator, replacing RoPE in MLA.

Keeping MLA as full attention is an interesting choice, as all other open weight models move to Grouped Query Attention (GQA). MLA uses an absorption trick to reduce the computation of a decode step at the cost of extra computation during the prefill step. This is a sensible trade-off for decode-dominant reasoning workloads; for prefill-dominant agentic workloads, extra computation becomes a high cost with little benefits. As a result, all frontier open weight models use GQA-based attention mechanisms: GLM 5.2 DeepSeek Sparse Attention, DeepSeek V4 Compressed Sparse Attention, MiniMax M3 MiniMax Sparse Attention, and MiMo V3 HySparse attention based on GQA. We suspect Moonshot's future models such as Kimi K4 will feature attention mechanisms that replace MLA.

SemiAnalysis is a reader-supported publication.

To receive new posts and support my work,
consider becoming a free or paid subscriber.

Subscribe

KV Cache Efficiency

We argue that **one should not infer KV cache efficiency solely based on KV cache space complexity**. KV cache size is not a standalone factor but a property of the model design: no open weight models are released with static KV cache compression techniques, and model architecture inference efficiency affects KV cache efficiency. The effects of KV cache size also vary, depending on the total memory capacity.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

architecture system efficiency and the KV cache size to understand the KV cache efficiency, and we quantify that with **KV throughput**.

KV Throughput

KV throughput is defined as KV cache size divided by the prefill time (Time to token), given a specific sequence length. KV throughput represents the minimum bandwidth required to reliably serve a model with PD disaggregation, but it is a good proxy for understanding KV cache efficiency. Prefill time encapsulates the efficiency of the model architecture, and as the sequence length increases, we will see the memory-bounded and the compute-bounded situations. As shown in the table below, we can see the benefits of hybrid linear attention become more pronounced as sequence length increases.

Seq Len	Hybrid				Dense	
	Kimi Linear	MiMo-V2-Flash	Qwen3.5-397B	Ring-2.5-1T	MiniMax-M2.5	Qwen3-230B
1K	1.19	0.82	4.13	7.27	4.94	4.12
8K	2.29	2.85	6.28	4.47	32.87	22.42
32K	3.87	4.66	8.25	2.59	59.93	33.39
128K	4.88	4.71	7.47	1.46	47.82	21.50

Source: [Prefill-as-a-Service: KVCache of Next-Generation Models Could Go Cross-Datcenter](#)

This is also a good way to understand the bandwidth requirements of KV cache offloading to different memory tiers in a cluster.

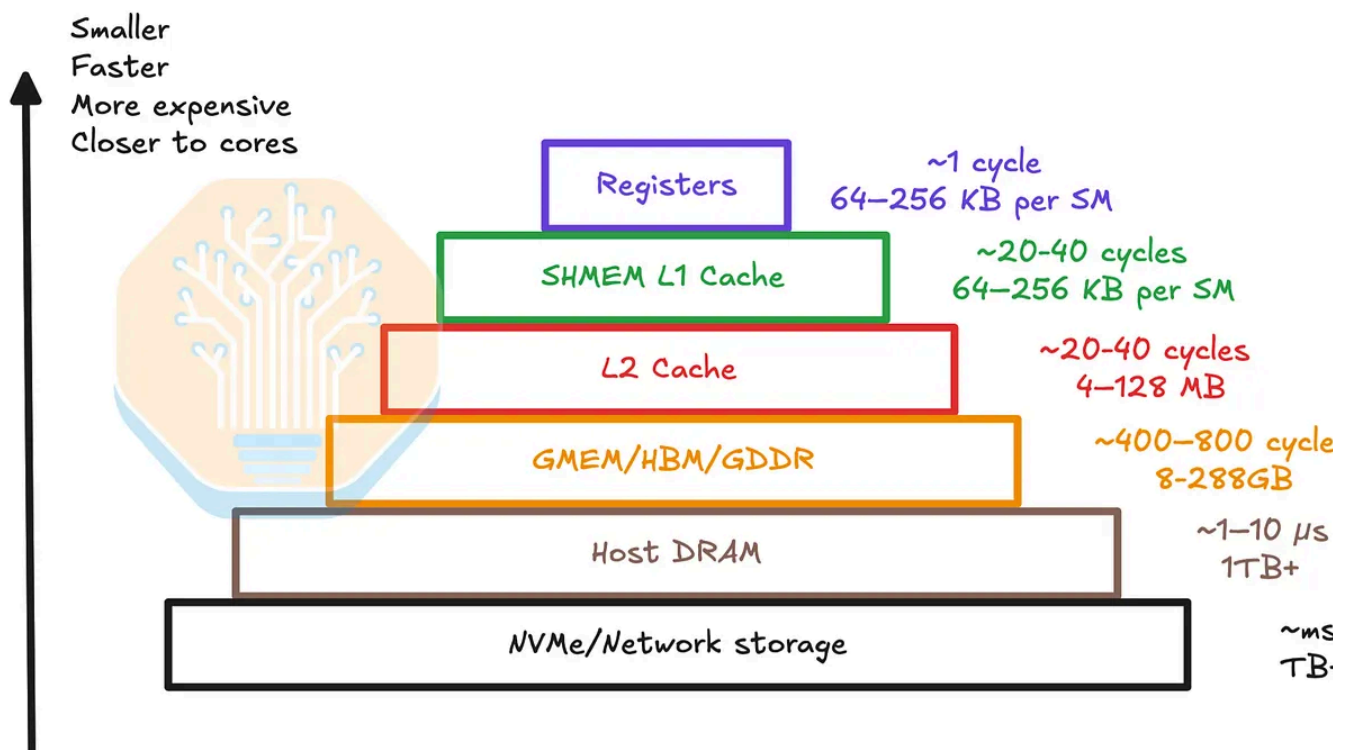
KV Cache Residency

The location where KV cache is stored follows the memory hierarchy. First, KV cache resides in HBM, the fastest memory in a GPU cluster, consuming whatever capacity is left by model weights and activations. As KV cache size exceeds the HBM capacity, it starts to spill over to slower memory tiers.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете [принять](#), [отклонить](#) их или [изменить свои настройки](#). См. нашу [политику конфиденциальности](#).

The analogy continues for memory coherency. Popular distributed KV cache framework Mooncake Store supports write-through and write-back policies for cache loading. Mooncake Store features a distributed KV cache pool that makes a cache visible to all workers. Implementing write-through policy between DRAM and the lower-level distributed KV cache pool offers multiple benefits in multi-scenarios, including sharing prefix cache across nodes, avoid KV cache duplication in tensor parallelized MLA, and KV cache redundancy when a node goes down.



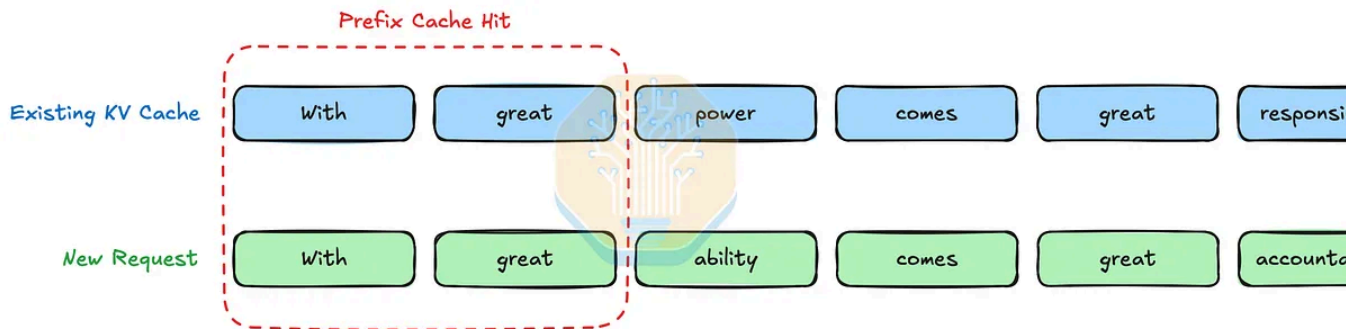
Source: SemiAnalysis

KDA Prefix Cache Management

At each token position in a request, Kimi K3 KDA's recurrent state is fixed in memory, whereas standard attention KV cache grows with sequence length. This KV cache space reduction comes at the cost of complicating prefix caching, especially in multi-scenarios.

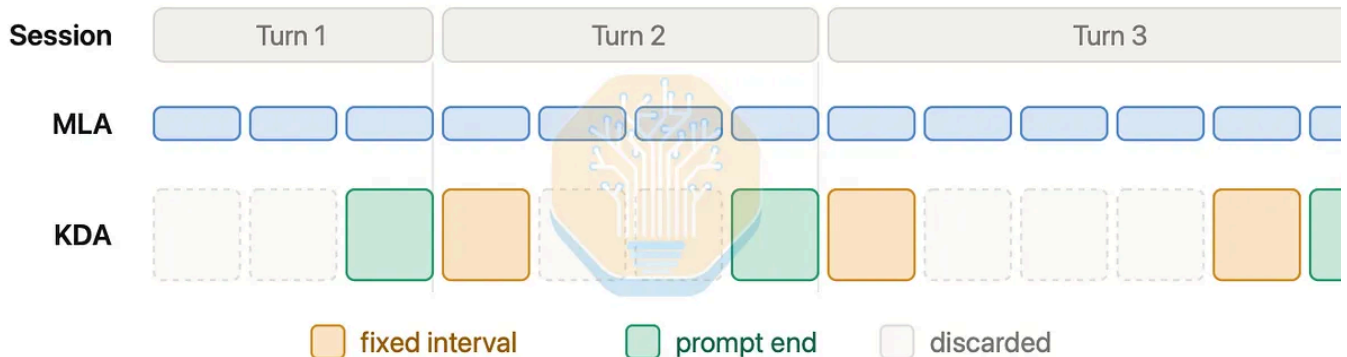
Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).



Source: SemiAnalysis

Identifying the longest prefix becomes a problem for linear attentions like MLA. Without prior knowledge of where the boundary of a prefix is, we will have to cache the KDA's recurrent state at every token position. This means every token has a cache, and the KV cache memory usage regresses to growing with sequence length, defeating the purpose of using linear attention. To tackle this problem, Moonshot saves recurrent states at a coarse granularity, e.g. vLLM caches every 32K tokens. vLLM additionally caches at prompt boundaries, since for agentic workloads, a new turn typically starts at the end of a prompt.



Source: [Kimi K3 Is Here: Efficient Day-0 Support on vLLM](#)

This shows that even though linear attentions like KDA greatly reduce KV cache memory consumption, **realistically during serving, they do not consume a constant amount of KV cache memory.**

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Residual connections are one of the key innovations that allowed us to build bigger deep neural networks through scaling model depth. The deeper the neural network the more expressive they become but training them naively is hard. Signals from the earlier layer need to be preserved till the last layer and gradient need to survive from output to first without vanishing.

Instead of modeling whole networks as a single function, passing information only through nonlinear transformations, residual networks connect smaller blocks with identity paths. Each block f_l learns a change to its input x_l , given by the recurrence

$$x_{l+1} = x_l + f_l(x_l)$$

The identity mapping allows features to carry from shallower units to any deeper unit and gives the gradient a path highway so they do not vanish.

$$\frac{\partial x_{l+1}}{\partial x_l} = I + f'_l(x_l)$$

While residual connections allow us to build deeper networks, they come with challenges.

Early layers heavily influence residual stream to have effect on final output. Because which residual stream has irreversible information loss with increasing depth. Later layers increase output gain to have effect on this modified residual stream which can destabilize training. Another variant like highway networks allow gating mechanisms for information flow but they suffer from the same crucial problem. Layers don't have selective access to information from earlier layers.

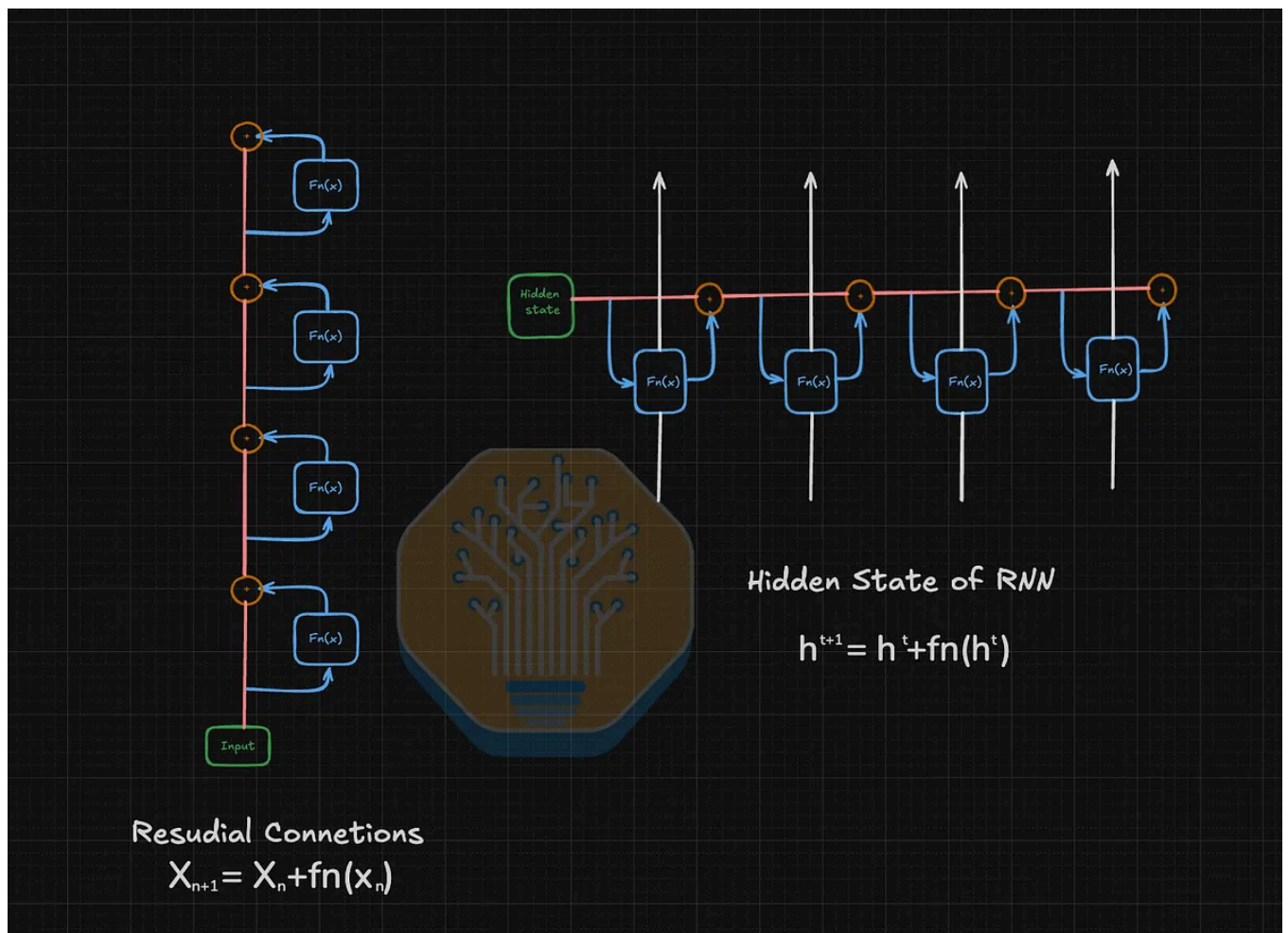
Recurrence In Time and Depth

Sequence modeling dominated by recurrent neural networks had the same recurrence formulation.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

signal.



Source: SemiAnalysis

Attention machines transformer removed this constrained by retrieving any token past with powerful and expensive attention mechanism

Attention on residual stream

Motivated by attention mechanism in sequence modeling, kimi developed attentic residual, where they take attention over depth blocks,

$\phi(a_i, k_i)$

Политика использования cookie-файлов

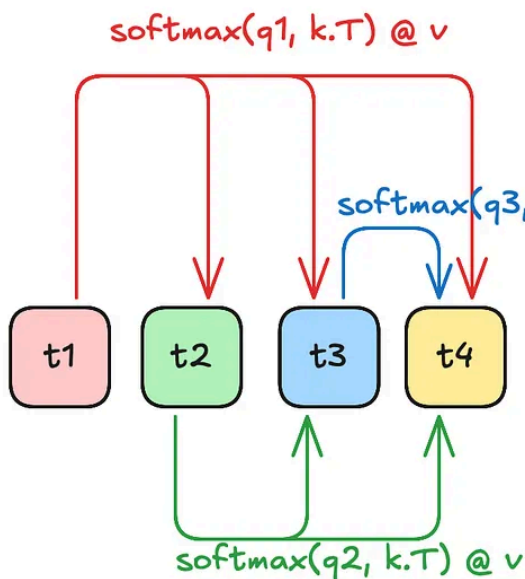
Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Standard causal self-attention computes the output of token t as a weighted sum of previous token representations:

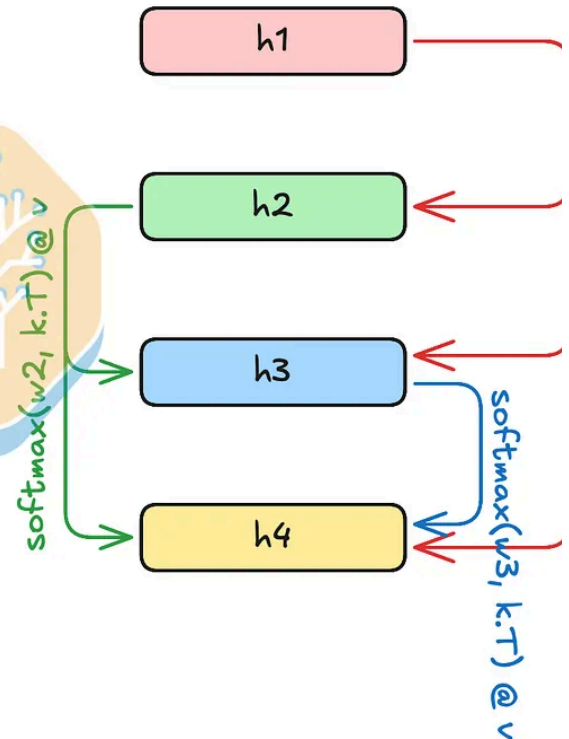
$$\mathbf{o}_t = \sum_{i=1}^t \alpha_{i \rightarrow t} \mathbf{v}_i, \quad \alpha_{i \rightarrow t} = \frac{\phi(\mathbf{q}_t, \mathbf{k}_i)}{\sum_{j=1}^t \phi(\mathbf{q}_t, \mathbf{k}_j)}$$

Attention Residuals use the same attention mechanism, but replace the sequence dimension with the depth dimension. Instead of attending over previous tokens, each layer attends over representations produced by previous layers.

Sequence-wise Attention



Depth-wise Attention



Source: SemiAnalysis

Unlike standard attention, the query is a learned parameter for each layer rather than being generated from the current token.

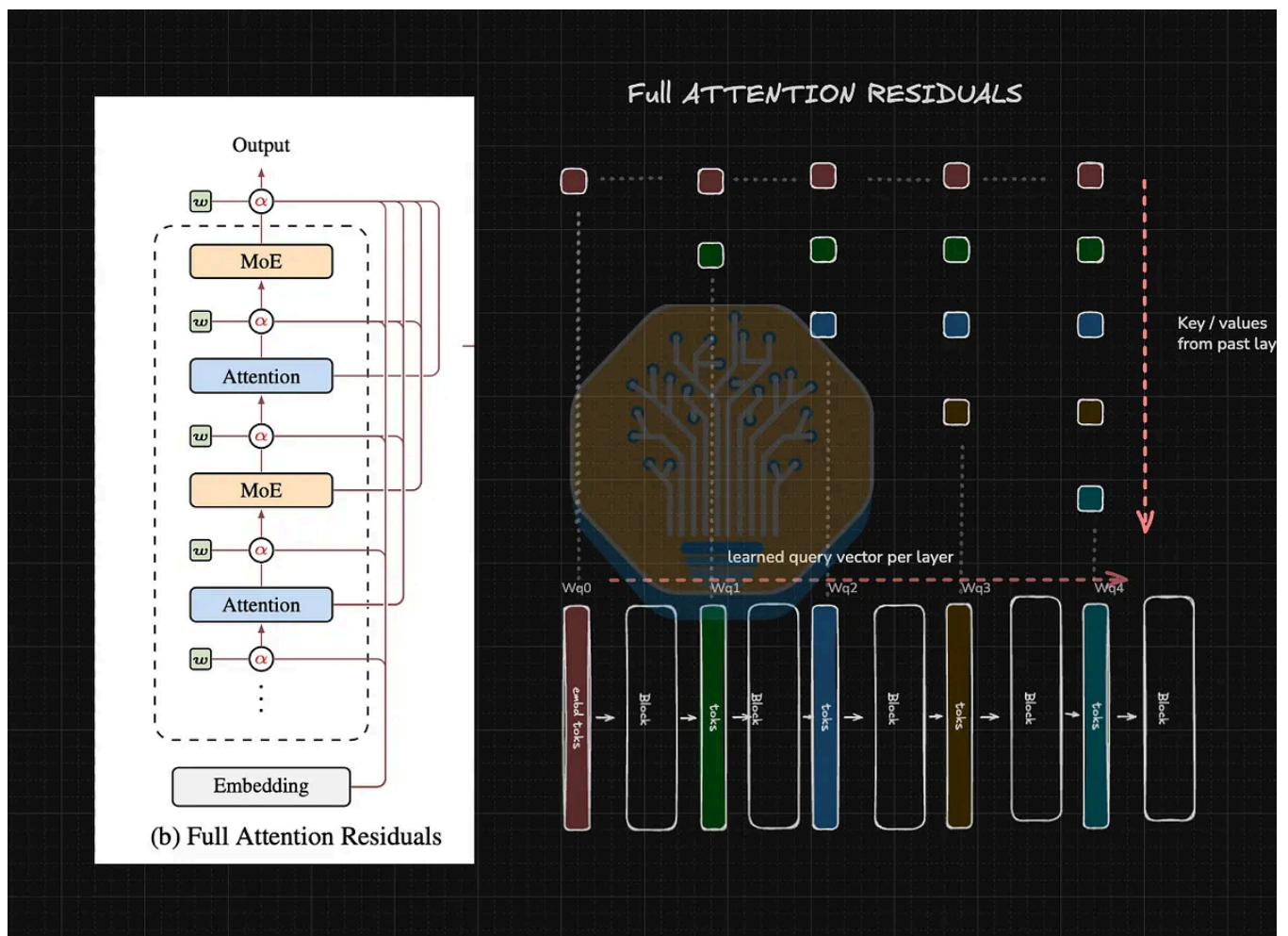
Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

For each layer ℓ , we define:

$$\mathbf{q}_\ell = \mathbf{w}_\ell, \quad \mathbf{k}_i = \mathbf{v}_i = \begin{cases} \mathbf{h}_1, & i = 0, \\ f_i(\mathbf{h}_i), & 1 \leq i < \ell. \end{cases}$$

Each colored block represents the output token representation from a previous transformer layer. Just as standard causal self-attention performs softmax attention over tokens in the sequence, Attention Residuals perform softmax attention over token representations produced by previous layers.



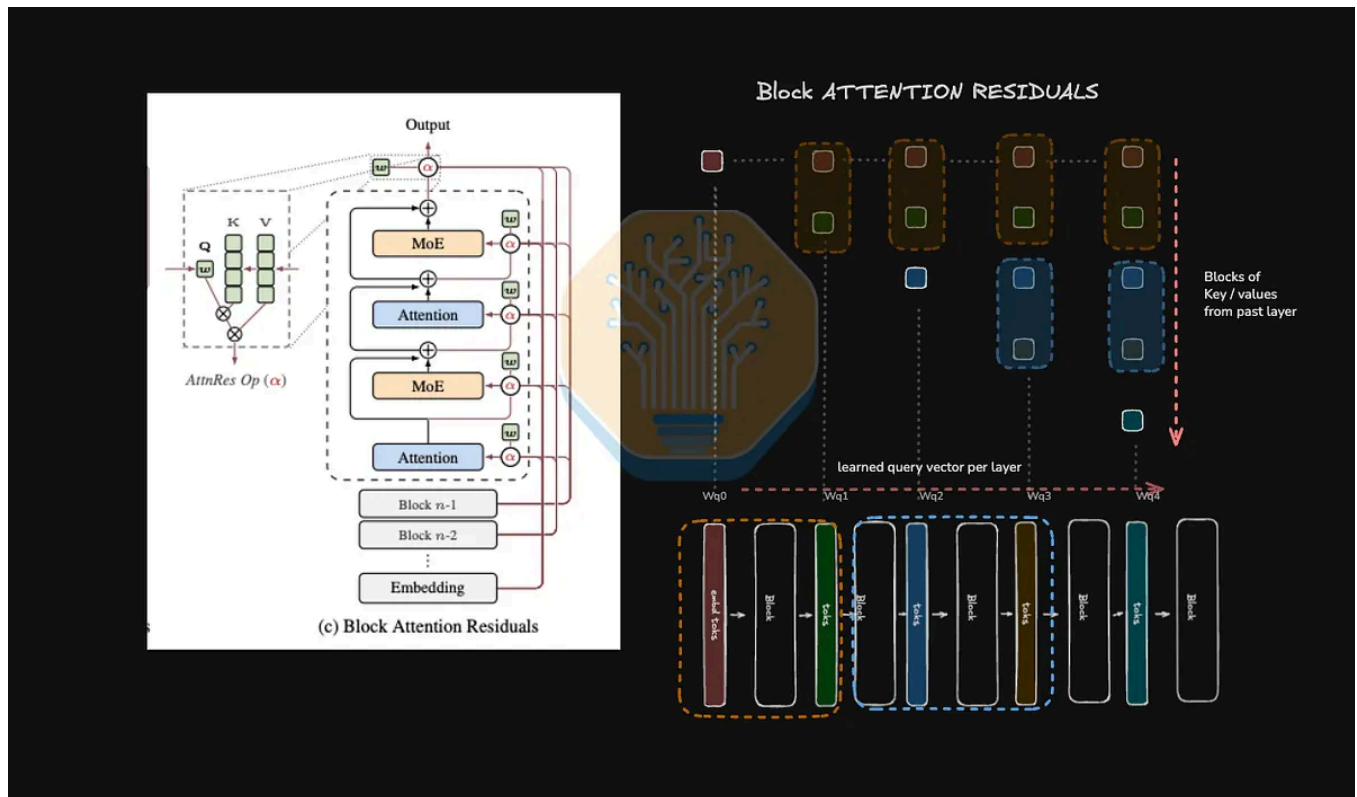
Source: SemiAnalysis

Attention residual allows the model to get fine grained control over what inputs to pick from past layers making the model more expressive

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Attention residual need to all past layer outputs for attention. For large models distributed over many GPUs this creates $O(Ld)$ communication overhead. To overcome this block attention residual dividends layers L into N blocks of S layers. Block AttnRes applies attention over completed block outputs and for current block its evolving partial sum.



source: SemiAnalysis

Block Attention has minimal tread over full attention residual but they cut down communication from $O(Ld)$ to $O(Nd)$.

Let b_n^i denote the partial sum over the first i layers in block n , such that

$$b_n = b_n^S, \quad b_0 = h_1.$$

For the i -th layer in block n , the available block representations are

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

$$\mathbf{q}_l = \mathbf{w}_l$$

Attention weights over the available block representations are computed as

$$\alpha_l = \text{softmax}(\mathbf{K}_l \mathbf{w}_l).$$

The output is the weighted sum of previous layer representations

$$\mathbf{h}_l = \alpha_l^\top \mathbf{V}_l.$$

Rather than depending only on the residual stream to preserve information, Attention Residuals give every layer direct, selective access to earlier representations. This block-based variant of attention residuals greatly reduces communication overhead while having competitive performance.

Block residuals show better scaling compared to standard residual connections, achieving 1.25× compute efficiency. Consistently lower validation loss compared to baseline gap widening with decay phase. Unlike standard residual networks where output magnitude increases as depth increases, selective aggregation of block attention has a bounded output. And consistent gradient magnitude.

Training

Unlike standard residual networks, attention residuals need all N-1 block input for computation of the Nth layer. This becomes a problem for pipeline parallelism as layer blocks output need to be transferred across stages.

With clever cross stage caching and activation checkpointing, Kimi reduced overhead to only 4% compared to standard architecture for pipeline parallelism.

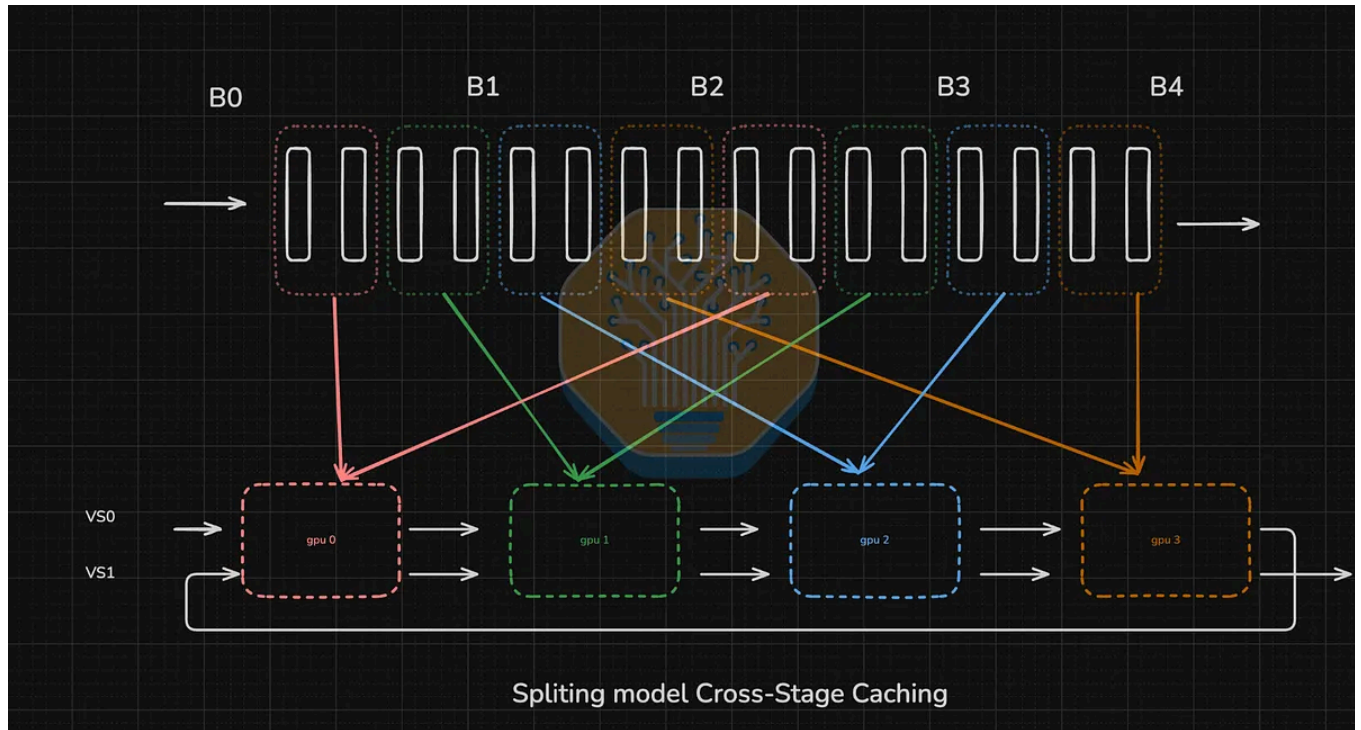
Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

This is quadratic cost growth for each physical and virtual stage

$$\text{Comm}_{\text{naive}} = \sum_{j=1}^{C-1} j N_p \cdot d = \frac{C(C-1)}{2} N_p d$$

This high communication can be reduced by caching input across virtual stages. Blocks computed in earlier layer can be stored in local memory,

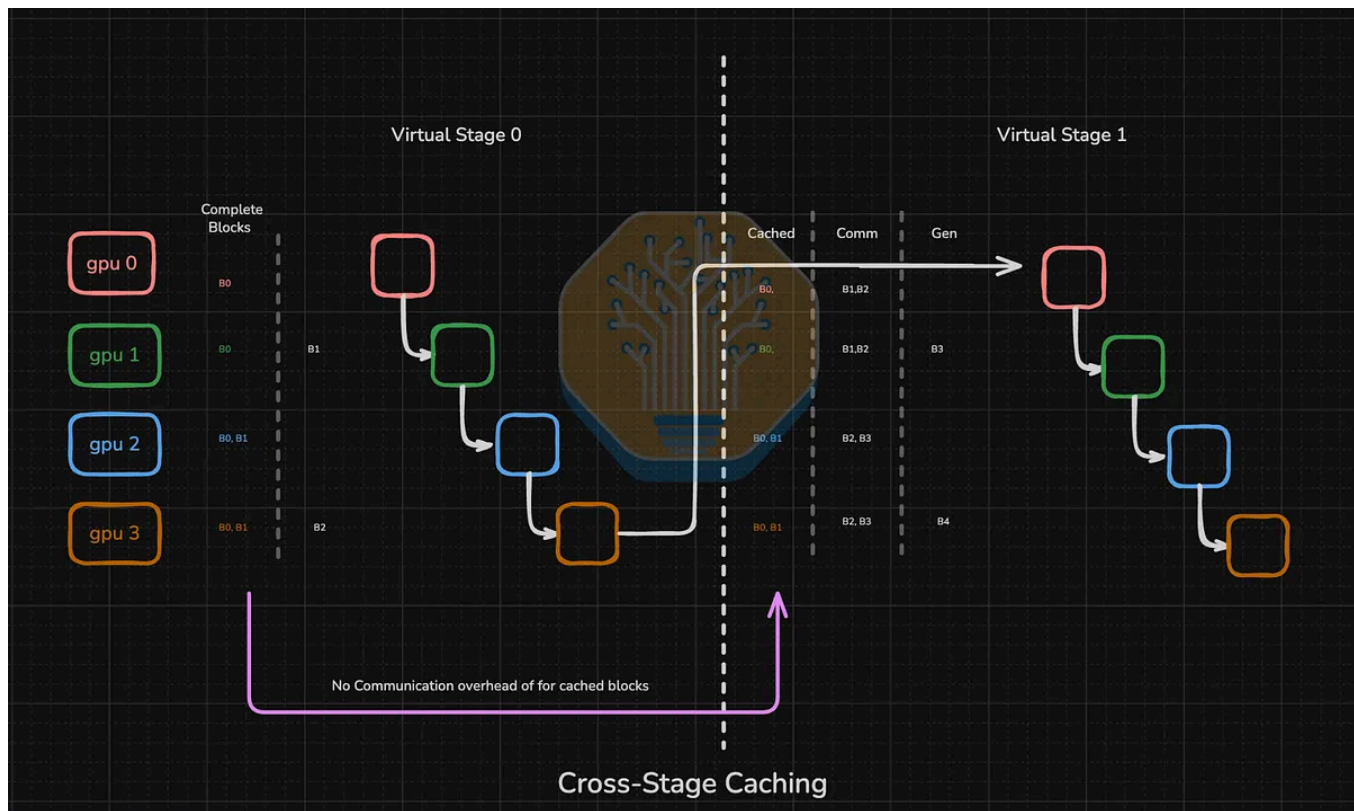


Source: SemiAnalysis

For the first virtual stage all block embedding needs to be transferred in the physical stage, each completed block is stored on respective rank. For all subsequent virtual stages all cached blocks can be reused for computation. Only the block not present on the rank need to be transferred for attention to residual computation.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).



Source: SemiAnalysis

These split communication costs for first and subsequent virtual stages. For the first virtual stage it needs to incur the same quadratic cost for all physical layers. In subsequent virtual stages we need cached inputs from local devices and transfer only PNP chunks needed cutting down total communication from $O(C)$ to $O(P)$

$$\text{Comm}_{\text{cached}} = \underbrace{\frac{P(P-1)}{2} N_p d}_{\text{first virtual stage}} + \underbrace{(V-1) P^2 N_p d}_{\text{subsequent virtual stages}}$$

The cutdown of communication is directly proportional to virtual stages V . Because this for full stage of one forward and backward pass all computation and communication can be overlapped

Memory overhead

Due to cross stage caching all blocks are stored once across all V virtual stages. We

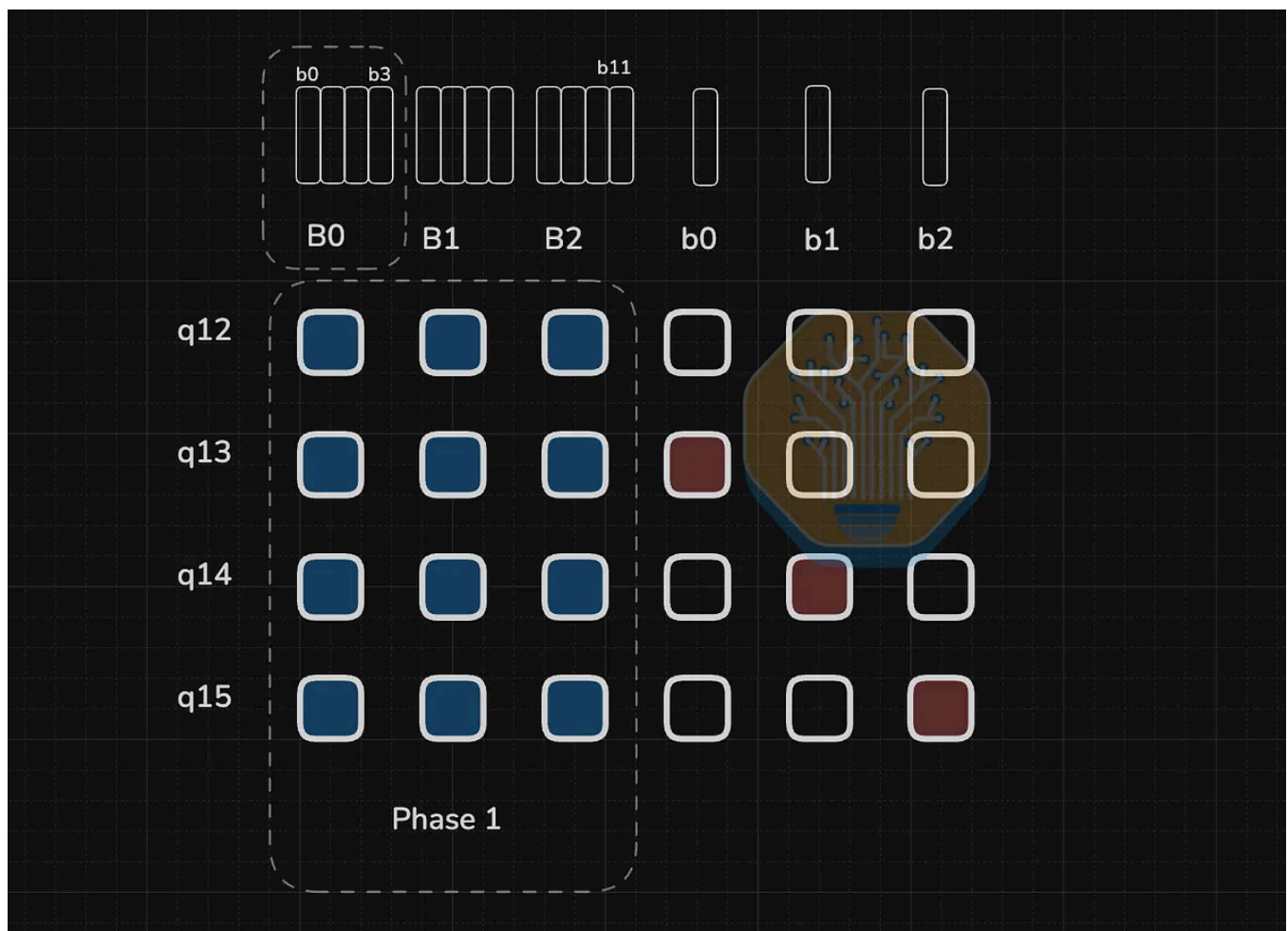
Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Inference

Because Attention Residuals need the output of all previous blocks to compute attention, a naive implementation has excessive memory accesses. To reduce overlap, inference is split into two phases which mirror prefill and decode stages of autoregressive attention. This computation is divided into inter-block attention for completed blocks and intra-block attention for evolving attention in the running block.

Phase 1: Parallel Inter-Block Attention

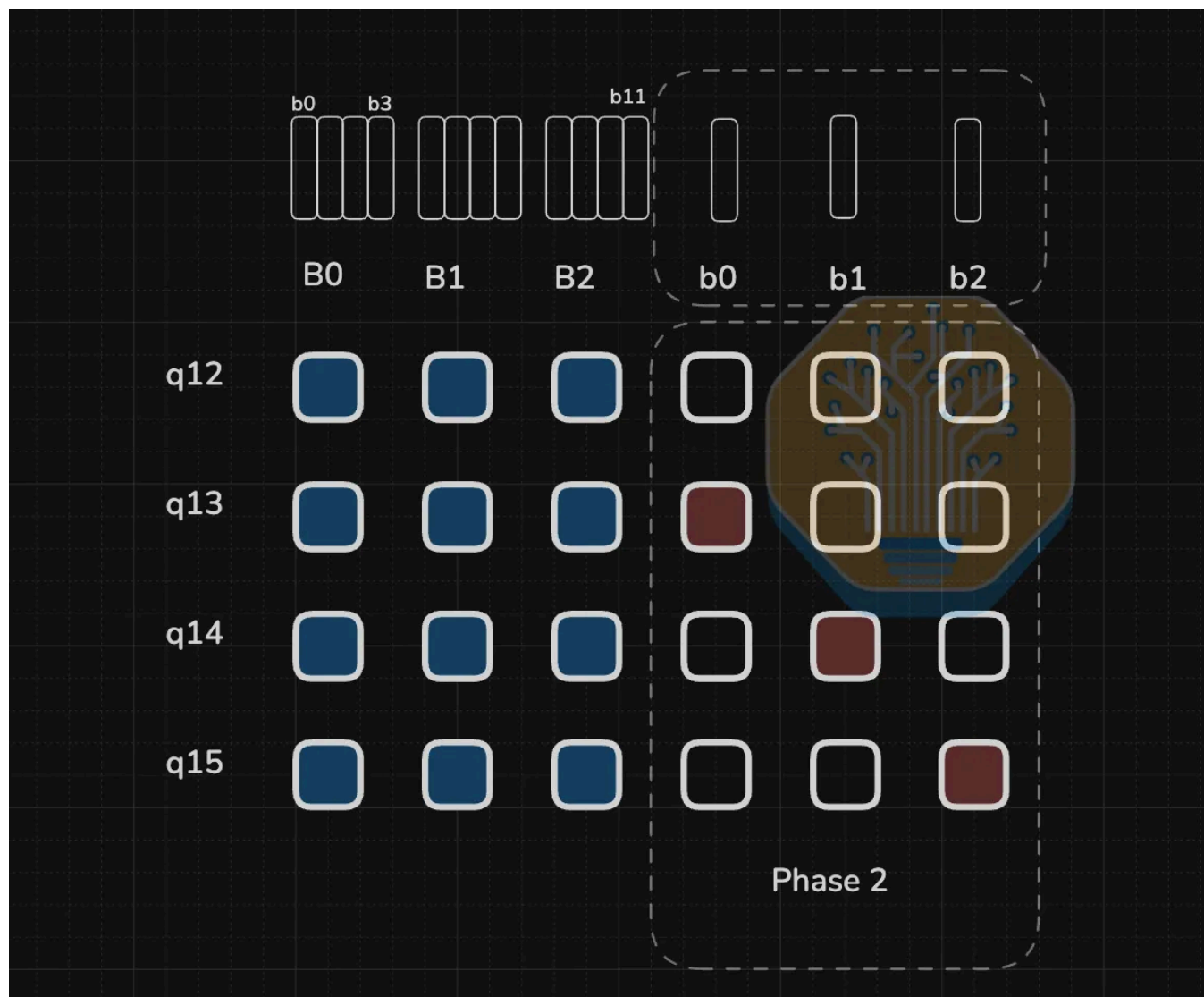


Source: SemiAnalysis

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Phase 2: Sequential Intra-Block Attention



Source: SemiAnalysis

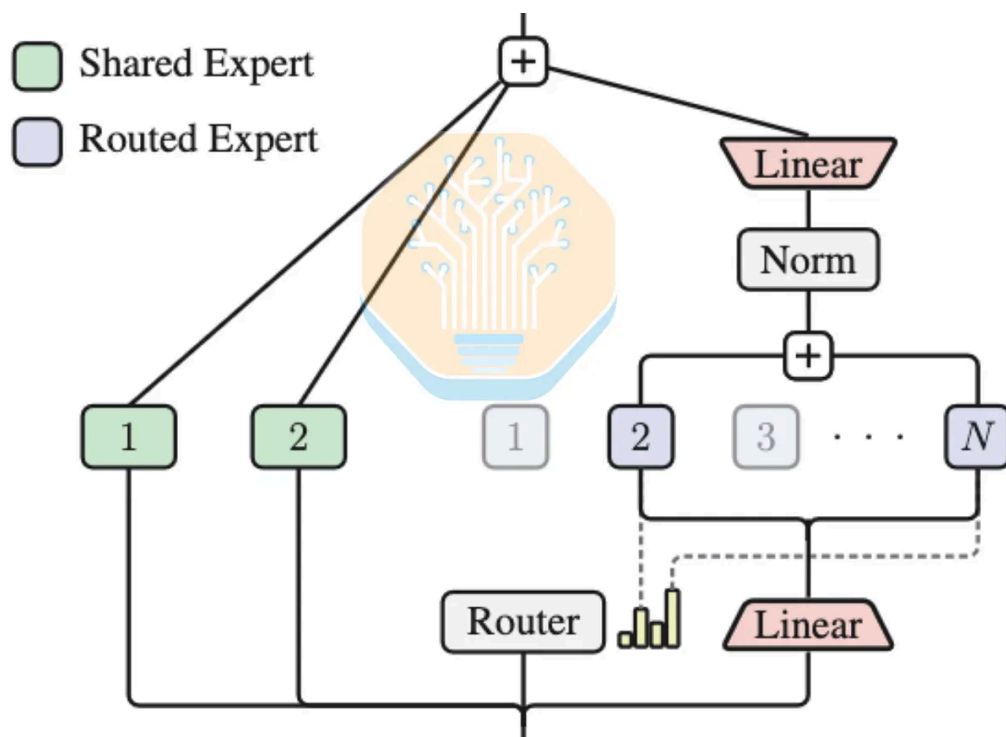
This phase is analogous to the decode phase, Similar to flash attention, evolving so it can be computed with online softmax for intra blocks combined with precompute inter block results. Which reduces redundant memory access.

With this two phase design, the IO footprint is similar to standard residual architecture, with only the addition of phase ones inter block computation, amortized by batching all queries in the block.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

LatentMoE compresses the routed tokens before the dispatch operation, and then decompresses them after the aggregation operation. In Kimi K3's Stable LatentMoE they apply an RMSNorm before the up-projection (decompressing) operation to re-sensitivity to scale variations and improve model performance.



Source: [Kimi K3 Tech Report](#)

Here we explain the design principles behind LatentMoE regarding MoE communication. As shown in the LatentMoE paper, the communication volume is proportional to total routed tokens t , number of active experts K , and expert input dimension d , while being inversely proportional to the expert parallel size E . This potentially the reason behind Kimi K3's latent MoE dimension size and active expert count configuration. Kimi K2 series feature 8 active experts with input dimension 7168, so Kimi K3's latent input dimension size being 3584 (half of 7168) would allow the active expert count to double to 16 without increasing the communication volume.

However, the **ratio of communication to computation time** is arguably more important.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

dimension size would decrease the ratio, meaning that the theoretical maximum fraction of communication that can be hidden is higher. Here we derive the formula

- t : total input tokens across the expert parallel (EP) domain
 - K : number of active experts per token
 - N : number of total experts
 - E : Ranks in the EP domain
 - d : Expert input dimension
 - m : Expert intermediate dimension
 - P : Aggregate bytes communicated per activation element (dispatch + combine)
 - F : Effective FFN expert (modeled as SwiGLU) computation throughput per GPU in FLOP/s
 - B : Effective uni-directional network bandwidth per GPU, B/s
1. Assuming uniform expert routing, each GPU is assigned $t * K / E$ tokens
 2. Assuming uniform expert routing, an average $1 / E$ tokens are local to the source GPUs, so each GPU dispatches $(t * K / E) * (1 - 1 / E)$ tokens
 3. Each token is a d dimensional vector, so the communication volume per token is $d * P$
 4. The communication volume per GPU is $(t * K / E) * (1 - 1 / E) * d * P$
 5. The communication time $T_{comm} = (t * K * d * P) / (E * B) * (1 - 1 / E)$
 6. The SwiGLU computation involves 3 matrix multiplications:
 - a. Up (First) projection: d to m
 - b. Gate projection: d to m
 - c. Down (Second) projection: m to d

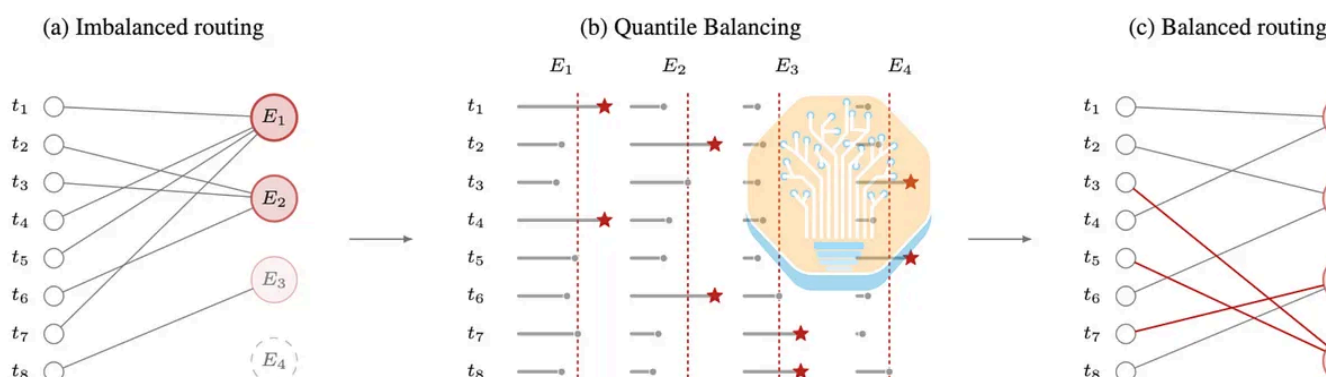
Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

$$\begin{aligned}
& T_{comm} / T_{comp} \\
&= ((t * K * d * P) / (E * B) * (1 - 1/E)) / ((6 * d * m) * (t * K / E) / F) \\
&= (P * F) / (6 * m * B) * (1 - 1/E)
\end{aligned}$$

We believe this formula also motivates an increase in expert intermediate dimension to 3072 in not just Kimi K2 to K3, but all recent open weight models, including DeepSeek V4 Pro, MiniMax M3, MiMo V2.5 Pro, and Inkling. As hardware improves and expert weight precision reduces to save memory capacity, the compute throughput increases, so one way of reducing the ratio is by increasing the expert intermediate dimension.

Quantile load balancing (QB)



Source: [Kimi K3 report](#)

Many previous load balancing methods require careful hyperparameter tuning. Quantile balancing is hyperparameter free aux-loss free load balancing technique developed by Jianlin Su in [Feb 2026 blog post](#)

Base principle QB is the same as auxfree load balancing where router biases are updated dynamically based on the system's load. But instead of updated bias by so small coefficient like aux-free lb, QB directly computes the next bias from the

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

Algorithm 1: The alternating QB solver.

Input: score matrix $s \in \mathbb{R}^{m \times n}$
Output: assignment $x \in \{0, 1\}^{m \times n}$

- 1 Initialize $\beta = \mathbf{0}_{1 \times n}$;
- 2 **for** $t = 1, 2, \dots, T$ **do**
- 3 $\alpha \leftarrow \text{desc_sort}(s - \beta, \text{axis}=1)_{[:, k:k+1]}$
- 4 $\beta \leftarrow \text{desc_sort}(s - \alpha, \text{axis}=0)_{[mk/n: mk/n+1]}$
- 5 **end**
- 6 **return** x with $x_{i,j} = 1$ if $j \in \text{argtop}_k(s_i - \beta)$, and 0 otherwise

Source: [Kimi K3 report](#)

QB tries to find the bias that would have approximately balanced under the current cutoffs and routing on the current batch, solving constraint optimization problem and applying these updates for the next batch. The first constraint is that each token routed to exactly k experts. The second constraint is a batch of m tokens each pick experts, gives (mk) assignments in total, to spread load evenly across n experts each expert should process $q = mk/n$ tokens.

Each token finds the cutoff threshold as the $(k+1)$ -th highest biased router score and uses it to calculate the bias update needed to balance load for each expert. For each expert, QB sorts the margins between its router score and every token's cutoff. It sets negative bias to $q+1$ the largest margin, leaving exactly q margin above the threshold. Since $q/m = k/n$, this is $(1-k/n)$ quantile of the margin, which is why it's called Quant Balancing.

SemiAnalysis is a reader-supported publication.

To receive new posts and support my work,
consider becoming a free or paid subscriber.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

As of 30th July, all providers on OpenRouter have a floor of \$3 per million tokens input and \$15 per million tokens output. Both Nvidia and AMD had Day 0 recipes on vLLM boasting DRAM offload and DSpark speculative decoding.

Standard		Latency / throughput		P50	Filter quantization	
Provider	Input /M	Output /M	Cache Read /M	Latency	Throughput	Up
Modal	\$3.00	\$15.00	\$0.30	2.88s	42 tps	96%
Baseten	\$3.00	\$15.00	\$0.30	1.80s	22 tps	99%
Morph	\$3.00	\$15.00	\$0.30	1.20s	31 tps	96%
Fireworks	\$3.00	\$15.00	\$0.30	2.14s	37 tps	98%
Together	\$3.00	\$15.00	\$0.30	2.72s	33 tps	95%
Moonshot AI	\$3.00	\$15.00	\$0.30	4.81s	22 tps	99%
Fireworks Fast	\$4.50	\$22.50	\$0.45	1.27s	78 tps	97%
DigitalOcean	\$3.00	\$15.00	\$0.30	3.40s	34 tps	89%

Source: [OpenRouter](#)

On InferenceX, we benchmark Kimi K3 serving performance directly on recorded internal claude code traces. We replay an hour of these traces as they reach a steady state. There is a median of 142k input tokens and a median of 444 output tokens per turn with a median of 65 turns per session. The short output tokens per turn is typical for workloads on agentic harnesses, where the agent calls tools frequently, even embedded tool uses.

This benchmark is a big step up from our previous 8k1k/1k1k benchmark, as it truly reflects real-world agentic use cases. From a systems perspective, it is also realistic and closest to production systems. It can reflect KV cache behavior, including prefill cache and KV offloading to DRAM.

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

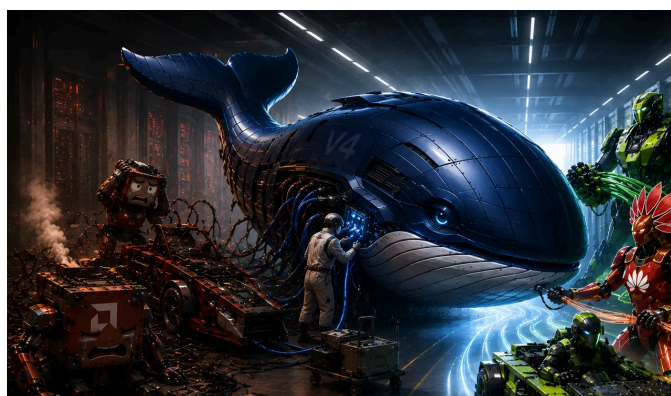


Source: [InferenceX](#)

For Kimi K3, Day 0 bringup was easier than DSv4 due to better documentation and preparation ahead of weights release. Appropriate images and a speculative decode model were released at the same time as the weights.

DeepSeekV4 1.6T Day 0 to Day 43 Performance Over Time - Huawei, GB300 NVL72, MI355X, B200

BRYAN SHAN, CAM QUILICI, AND 5 OTHERS • 9 ИЮН.

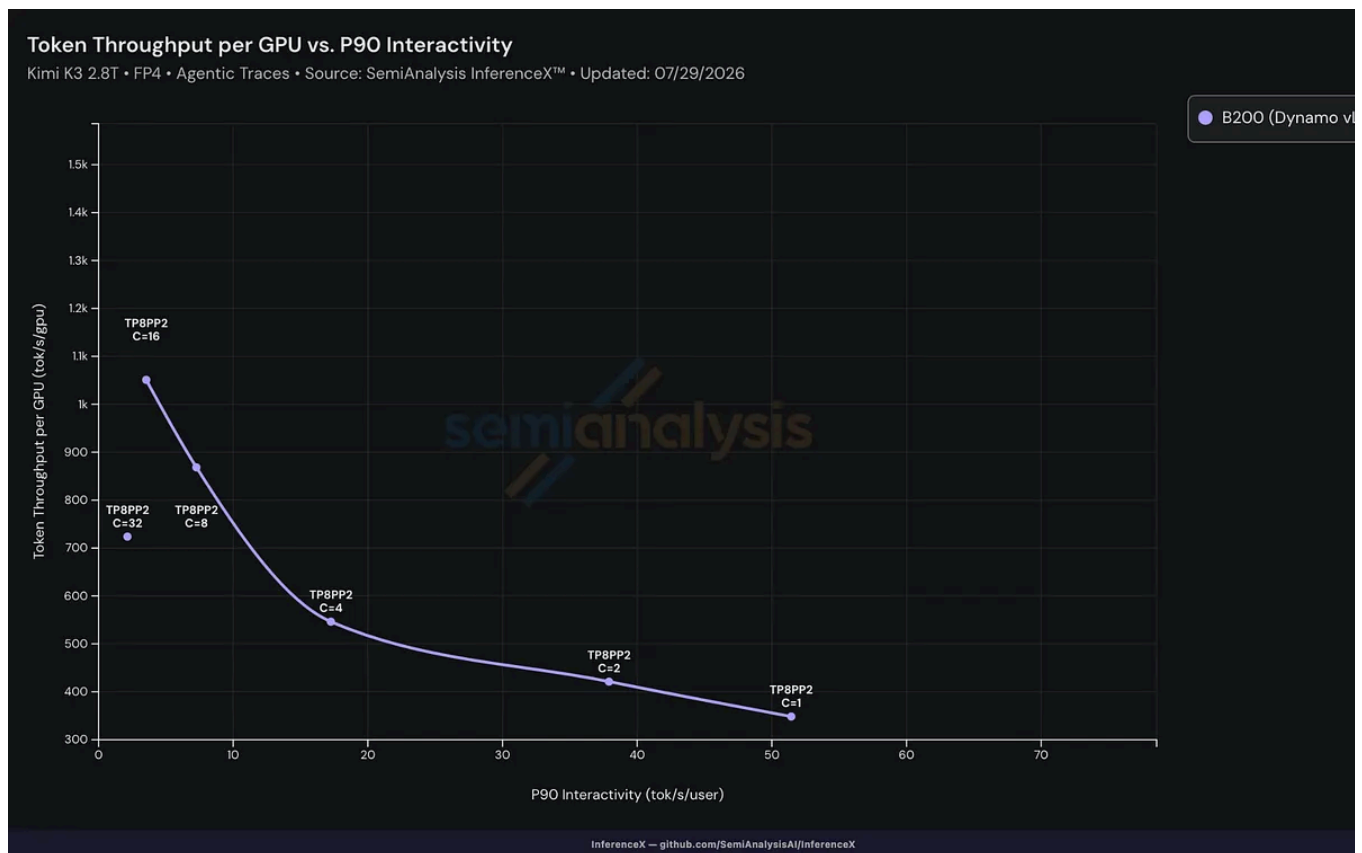


The release of DeepSeek v4 marks another step forward for the open model community - unsurprisingly, it is the product of a Chinese lab. The evolution of its performance over time is of paramount importance to the AI Ecosystem. The open-source InferenceX engineering team has pulled multiple all-nighters to measure performance results for this model on Day 0, Day 1, Day 2, and

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).

PP.



Source: [InferenceX](#)

For B300, the model fits on 1 node and serves well. After accounting for the weight GPU HBM can only hold 3.25M tok. In the graph below, throughput goes up as batch sizes increase until concurrency increases above 8. This roughly correlates to the 3.25M tok KV cache budget, and cache starts to thrash, resulting in hit rates falling < 10% when theoretical hit rate is 95%.

This post is for paid subscribers

[Subscribe](#)

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).



A guest post by

Shubham Choudhari

hi

Subscribe to Shubh

© 2026 Dylan Patel · [Privacy](#) · [Terms](#) · [Collection notice](#)

Substack is the home for great culture

Политика использования cookie-файлов

Мы используем файлы cookie, чтобы улучшить взаимодействие с сайтом, а также для аналитики и маркетинга. Вы можете принять, отклонить их или изменить свои настройки. См. нашу [политику конфиденциальности](#).